The Hashemite University

Faculty of Prince Al-Hussein Bin Abdullah II for Information Technology

Department of Computer Information Systems

Zaytoun

A Web Platform for Recipe Discovery,
Arab Culinary Heritage, and Food Education

Graduation Project

Submitted in partial fulfillment of the requirements for the
degree of Bachelor of Science in Computer Information Systems

Submitted by: **Faris Hamdan**

Student ID: **[Insert Student ID]**

Supervisor: **[Insert Supervisor Name]**

Academic Year 2025 / 2026

Zarqa, Jordan

Acknowledgement

All praise is due to Allah, who granted me the strength, patience, and clarity of mind to complete this work.

I extend my deepest gratitude to my supervisor, [Insert Supervisor Name], whose continuous guidance, constructive feedback, and academic generosity shaped this project at every stage. Their willingness to review the work in detail and to share their experience made a real difference in the quality of the final outcome.

I am grateful to the faculty of the Department of Computer Information Systems at the Hashemite University. Every course I attended contributed in some way to the technical foundation behind this project, from the early software engineering modules to the database, networking, and human computer interaction courses that informed the final design.

I would also like to thank the members of the examination committee for taking the time to review and evaluate this work.

My sincere thanks go to my family for their patient encouragement throughout my university years. Their support, both moral and practical, was essential during the long evenings of development and writing.

Finally, I thank my friends and classmates who tested early versions of the platform on their phones and offered honest observations about what felt natural and what did not. Their feedback shaped many of the small interaction details that, taken together, make the platform feel polished.

Faris Hamdan

June 2026

Undertaking

I, Faris Hamdan, holder of student ID [Insert Student ID], a final year undergraduate student in the Department of Computer Information Systems at the Faculty of Prince Al-Hussein Bin Abdullah II for Information Technology, the Hashemite University, hereby declare the following:

1. The work presented in this report, titled "Zaytoun: A Web Platform for Recipe Discovery, Arab Culinary Heritage, and Food Education," is my own original work, developed during the academic year 2025 to 2026.
2. The implementation, design, written content, and all source code submitted as part of this graduation project were produced by me under the academic supervision listed on the cover page.
3. Any third party libraries, public APIs, datasets, or open source assets used during development have been properly attributed in the references section of this report. Their use is consistent with the licenses under which they are distributed.
4. Any direct quotations, paraphrased material, or external information drawn from books, articles, websites, or other sources are clearly cited in the text and listed in the references.
5. No part of this work has been previously submitted for the award of any other academic degree at the Hashemite University or at any other institution.
6. I understand that any form of plagiarism, fabrication of results, or misrepresentation of authorship would be a serious breach of academic integrity and may result in the cancellation of this work and disciplinary action under university regulations.
7. I confirm that the project repository, the deployed website, and the supporting documentation submitted with this report represent the actual state of the system at the time of submission.

Signed:

Faris Hamdan

Date: June 2026

Certificate of Supervision

This is to certify that the graduation project titled:

"Zaytoun: A Web Platform for Recipe Discovery, Arab Culinary Heritage, and Food Education"

was carried out by the student:

Faris Hamdan, Student ID [Insert Student ID]

under my supervision, in the Department of Computer Information Systems, Faculty of Prince Al-Hussein Bin Abdullah II for Information Technology, the Hashemite University, during the academic year 2025 / 2026.

The student conducted the analysis, design, implementation, testing, and documentation work in accordance with the academic requirements of the department. The submitted report, source code, and deployed application reflect the student's own effort. The work is, in my opinion, of a standard that qualifies it for submission to the examination committee in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Information Systems.

I confirm that the project meets the academic and professional standards expected of a graduation project in the Department of Computer Information Systems.

_____

Supervisor: [Insert Supervisor Name and Title]

Department of Computer Information Systems

The Hashemite University

Signature: _____

Date: _____

_____

Department Chair: [Insert Chair Name]

Signature: _____

Date: _____

Abstract

Cooking content is abundant on the modern web, yet the experience of discovering reliable recipes, learning techniques, and exploring regional cuisines remains fragmented. Mainstream platforms are heavily oriented toward western kitchens, lean on aggressive advertising, and rarely treat Arab and Levantine culinary heritage as a first class domain. Mobile users in the Arab region in particular often face slow pages, intrusive popups, and search tools that do not understand Arabic input. This project, named Zaytoun, addresses these gaps by delivering a unified, fast, and editorial style web platform that combines recipe discovery, Arab cuisine, food films, a structured cooking academy, a markets atlas, a chef hall, a drinks library, and a short video feed in one cohesive product. The platform is built as a single page application using React 19, TypeScript, Vite, and Tailwind CSS, with TanStack Query for data fetching, Fuse.js for fuzzy search, jsPDF for client side document export, and a progressive web app layer for offline friendly behavior. Recipe data is sourced from the Spoonacular API, while curated content libraries are maintained as typed static datasets. A custom smart search component supports Arabic and English input, voice queries through the Web Speech API, recent and suggested queries, and a "did you mean" suggestion based on a relaxed fuzzy threshold. A fitness page computes daily macronutrient targets using the Mifflin St Jeor and Katch McArdle formulas and exports the results as PDF. The platform was deployed at zaytoun.online through Vercel and verified across desktop and mobile browsers. The result is a production grade Arabic and English food platform that is faster, cleaner, and more culturally relevant than the public alternatives reviewed during the analysis phase.

Keywords: web application, recipe discovery, Arab cuisine, React, TypeScript, fuzzy search, Arabic localization, progressive web app.

Table of Contents

List of Tables

List of Figures

Chapter 1. Introduction

1.1 Overview

Zaytoun is a web platform developed as a graduation project in the Department of Computer Information Systems at the Hashemite University. The project addresses the way users in the Arab region discover, learn about, and engage with food content online. Rather than producing yet another recipe list, the platform combines several content categories that are usually scattered across different websites into one cohesive product. These categories include practical recipes, regional Arab dishes, food films and documentaries, structured cooking lessons, an atlas of food markets around the world, a hall of notable chefs, a drinks library covering hot and cold beverages, and a vertical short video feed inspired by modern social platforms.

The platform is implemented as a single page application using a modern frontend stack centered on React, TypeScript, and Tailwind CSS. It uses the Spoonacular API for the main recipe catalogue, while curated content libraries are stored as typed static datasets that ship as part of the application bundle. Users access the platform through any modern browser at the production domain zaytoun.online, which is also installable as a progressive web app.

Zaytoun is designed to be visually clean, fast on mobile devices, accessible in both English and Arabic, and respectful of the user's time. There are no popups, no advertising scripts, and no third party tracking cookies. The interface follows a soft editorial style with generous whitespace, typography based on a system font stack, and a controlled color palette built around cream, ink, terracotta, and sage tones.

1.2 Project Motivation

Food is one of the most searched topics on the internet. According to several published analyses of search engine trends, recipe queries consistently rank among the most common categories of consumer search, especially during evening hours and weekends when people plan meals. Despite this volume, the websites that dominate global recipe search results share recurring problems. They are crowded with advertising, slow to load, and oriented almost exclusively toward western cuisines. Mobile users in the Arab region in particular experience these issues more sharply, because the pages are often heavier than the average mobile data plan can comfortably load.

Arabic content on these platforms is also limited. Famous regional dishes such as mansaf, mlukhiyeh, maqluba, and koshary are either missing, presented inaccurately, or buried beneath similarly named dishes. Where Arabic platforms do exist, such as Shahiya or Fatafeat, the focus is

narrower, the design has not been refreshed in years, and the search experience does not handle Arabic diacritics or transliterations well.

On a personal level, the motivation for this project also came from spending time in the kitchen during late nights and noticing how much friction there was between a vague idea (for example, "I want something fast with chicken and rice") and an actual cooking decision. Existing platforms forced the user through advertising heavy landing pages, long autobiographical introductions, and unrelated suggestions before reaching the recipe itself. The goal of Zaytoun is to remove that friction and present food content the way one would expect a well edited magazine to present it.

There is also an educational angle. Many young Arab users do not know much about classical regional dishes outside their own city, and the available online documentaries are scattered across YouTube channels with no curation. A platform that brings together recipes, regional context, films, and structured lessons in one place fills a real gap.

1.3 Problem Statement

The problems that this project addresses can be expressed in the following statements:

8. Existing recipe websites are visually overloaded, slow on mobile networks, and aggressive with advertising and pop ups, which damages the cooking experience instead of supporting it.
9. Major platforms do not give Arab cuisine the depth it deserves. Regional dishes from Jordan, Palestine, Egypt, the Levant, the Gulf, and North Africa are either absent or treated as exotic curiosities.
10. Search tools on these platforms do not understand Arabic input, do not handle transliteration, and rarely return useful results when a query contains diacritics or alternative spellings.
11. There is no widely used platform that combines recipes, regional cuisine context, food documentaries, structured cooking lessons, markets, chefs, drinks, and short videos in one continuous experience. Users must currently jump between Google, YouTube, Instagram, and Wikipedia to assemble that knowledge themselves.
12. Most cooking platforms do not include a built in tool for calculating daily macronutrient targets, which is increasingly important for users who track their nutrition.
13. Documentation, attribution, and editorial integrity around food content are weak on most aggregator sites, so users cannot easily evaluate the credibility of a recipe or a claim about a dish.

Taken together, these problems define a clear opportunity for a single, fast, editorial style web platform that respects the user, handles Arabic content properly, and treats food as a serious cultural domain.

1.4 Project Aim and Objectives

The aim of the project is to design, implement, and deploy a complete web platform that delivers a unified, fast, mobile friendly, and culturally aware food discovery experience, with first class support for both Arabic and English content.

To achieve this aim, the project sets the following objectives:

14. **Recipe discovery.** Provide a searchable, filterable recipe catalogue powered by a reliable external recipe API, with optional offline access to a curated subset.
15. **Arab cuisine library.** Maintain a typed, hand curated library of well known dishes from across the Arab world, each with ingredients, steps, and a verified video reference.
16. **Editorial content.** Provide curated libraries for food films, learning paths, food markets, chefs, drinks, and short videos that together form an editorial experience comparable to a quality food magazine.
17. **Smart search.** Implement a search component that supports fuzzy matching, Arabic and English input, voice queries, recent and suggested queries, and a "did you mean" fallback when no direct match is found.
18. **Bilingual interface.** Deliver a full English and Arabic interface, including right to left layout, an Arabic optimized font, and translated navigation.
19. **Fitness support.** Provide a macronutrient calculator that uses standard scientific formulas (Mifflin St Jeor and Katch McArdle) and exports its output as a PDF.
20. **Modern web standards.** Build the platform as a progressive web app, installable to the device home screen, with sensible caching of static assets and offline fallbacks.
21. **Custom domain and production deployment.** Publish the platform under a real custom domain (zaytoun.online) and verify it on real desktop and mobile devices, including devices on Jordanian mobile networks.
22. **Editorial standards.** Apply a single typography system, a controlled color palette, generous spacing, and a quiet interface that respects the user's attention. No advertising, no tracking scripts, no aggressive pop ups.
23. **Documentation.** Produce a complete academic report describing the analysis, design, implementation, testing, and conclusions of the project.

1.5 Project Functionality

The platform exposes the following major areas, each accessible from the global navigation and the home page:

- **Home.** Editorial landing page that introduces the platform, features the recipe of the day, and surfaces a curated trio of a learning path, a food film, and a chef.

- **Recipes.** Searchable and filterable catalogue of recipes, including dietary tags, cuisine tags, and a favorites system stored locally on the device.
- **Arab Cuisine.** A library of regional dishes from across the Arab world, including Palestinian, Jordanian, Egyptian, Levantine, Gulf, and North African dishes, each with full ingredients, steps, and a verified video reference.
- **Films.** A library of food documentaries and series, presented as a magazine grid with director, year, and synopsis. Each entry links to its video source.
- **Academy.** A learning section with curated lessons, structured learning paths, level metadata, and a progress tracker stored on the device.
- **Markets.** An atlas of food markets around the world, with location, specialty, and an embedded video tour.
- **Chefs.** A hall of notable chefs grouped by region, with nationality, signature dish, and a representative video.
- **Drinks.** A library of hot and cold drinks, including coffee, tea, sahlab, and traditional Arab beverages, each with ingredients and a video reference.
- **Shorts.** A vertical short video feed with scroll snap behavior, category filters, and an in app player modal, used for fun and quick discovery.
- **Fitness.** A page that contains a macronutrient calculator and curated fitness video channels, plus structured goal based plans.
- **Search.** A global smart search component, accessible from the header, that performs universal search across all libraries with Arabic, English, voice, and "did you mean" support.
- **Settings.** Theme toggle (light and dark), language toggle (English and Arabic), cookies preferences, and a clear local data action.

The features above form the core of the project. A high level platform map is given in Figure 1.1.

1.6 Project Results

By the end of the implementation phase, the platform was deployed to production at zaytoun.online and tested on multiple devices. The following high level results were achieved:

24. The recipe catalogue serves recipes through the Spoonacular API and falls back gracefully when the API is rate limited.
25. The Arab cuisine library contains regional dishes with full ingredients, steps, and verified videos.
26. The films library contains more than thirty food documentaries with verified video references.
27. The academy contains lessons grouped into learning paths.
28. The markets library covers more than twenty food markets around the world.
29. The chef hall contains forty chefs grouped into six regions.

30. The drinks library contains more than twenty drinks across hot and cold categories.
31. The shorts library contains thousands of verified short videos across forty categories, including the special "car eating" subcategory requested by the project owner.
32. The smart search component supports Arabic and English input, voice search, fuzzy matching, recent and suggested queries, and a "did you mean" fallback.
33. The macronutrient calculator computes daily targets and exports the results as a PDF.
34. The platform is installable as a progressive web app and works offline for already visited pages.
35. The platform is deployed under a custom domain, with a working SPA rewrite, working Open Graph metadata, and a working share image.
36. The user interface follows a single design system across all pages, with full Arabic right to left support.

1.7 Report Structure

The remainder of this report is organized as follows. Chapter 2 surveys existing food platforms and identifies the gaps that motivated Zaytoun. Chapter 3 presents the analysis and design of the system, including stakeholders, use cases, non functional requirements, constraints, and architecture. Chapter 4 details the implementation, the technology stack, and the problems encountered during development. Chapter 5 describes the testing and evaluation strategy. Chapter 6 concludes the report and outlines future work.

Chapter 2. Related Existing Systems

2.1 Introduction

This chapter reviews the most widely used recipe and food discovery platforms on the modern web, examines their strengths and weaknesses, and identifies the gaps that Zaytoun aims to fill. The review covers both global English language platforms and Arabic platforms that target the Middle East and North Africa region. The chapter ends with a comparison table and a description of the overall solution approach taken by Zaytoun.

The purpose of the review is not to dismiss existing platforms. Several of them are excellent products that have shaped the way users think about cooking content on the web. The review focuses instead on the specific aspects in which they fall short for the target audience of this project, which is a young Arab user with a mobile device, an unreliable mobile data plan, and an interest in both global and regional cuisine.

2.2 Existing Systems

2.2.1 Allrecipes

Allrecipes is one of the oldest and most popular community driven recipe websites in the United States. It contains millions of user submitted recipes with reviews and photographs. Its strengths include the size of its catalogue, the depth of its community ratings, and the broad coverage of common American dishes.

Its weaknesses are well known. The site is heavily monetized through advertising, including video pre rolls, banner ads, and interstitial overlays. Pages are slow to load on a typical mobile connection, often exceeding several megabytes per page. The catalogue is heavily skewed toward American cuisine, so dishes from Jordan, Palestine, or Egypt are either missing or represented by a single low quality submission. The search engine does not handle Arabic input at all and offers limited fuzzy matching for English queries with typos.

2.2.2 Tasty by BuzzFeed

Tasty is a recipe brand operated by BuzzFeed. It is best known for its short overhead cooking videos that became popular on social media. Its strengths are the visual style of its videos, the clarity of its recipe steps, and the strength of its content marketing on platforms such as Facebook and Instagram.

Its weaknesses are that it focuses on viral content rather than depth. Recipes are often simplified to the point of inaccuracy, and the platform pays very little attention to regional culinary heritage outside of a handful of "world food" features. The site is also tightly integrated with BuzzFeed's advertising and tracking infrastructure, which weighs the pages down. Arabic support is essentially absent.

2.2.3 Yummly

Yummly is a recipe aggregator that focuses on personalized recommendations. Its strengths include a clean visual style, an inventory based search ("what do I have in my fridge"), and integration with smart kitchen appliances.

Its weaknesses include heavy reliance on user account creation, a recommendation system that quickly becomes repetitive, and a strong bias toward American and European recipes. The platform also depends on a paid premium tier for some of its features, which limits its usefulness for casual visitors. Arabic content is rare and not well indexed.

2.2.4 BBC Good Food

BBC Good Food is a British recipe site with strong editorial standards and high quality photography. Its strengths include reliable testing of recipes, clear instructions, and a generally clean interface. It is one of the more trustworthy English language recipe destinations.

Its weaknesses are that it remains a regional brand. Coverage of Middle Eastern cuisine is shallow and often filtered through a British lens. Pages still load a number of advertising scripts. There is no Arabic version of the site.

2.2.5 Shahiya and Fatafeat

Shahiya and Fatafeat are two of the better known Arabic language food platforms. Shahiya is part of a regional digital media group and has a recipe catalogue oriented toward home cooks in the Gulf and Levant. Fatafeat is associated with the Fatafeat television channel and combines recipes with chef profiles and video content.

Their strengths include native Arabic content, regional dish coverage, and integration with regional television shows. Their weaknesses include outdated visual design, slow mobile pages, heavy advertising, and limited search capability. Neither platform offers a unified experience that combines recipes, films, lessons, markets, and chefs in the way Zaytoun does, and neither provides a smart search that fuses Arabic and English queries.

2.3 Overall Problems with Existing Systems

The review of the platforms above highlights a set of recurring problems:

Problem	Description	Affected platforms
Heavy advertising	Pages contain multiple ad scripts that slow loading and damage the reading experience	Allrecipes, Tasty, Yummly, BBC Good Food, Shahiya, Fatafeat
Poor mobile performance	Pages exceed several megabytes and are slow on regional mobile networks	Allrecipes, Tasty, Shahiya, Fatafeat
Weak Arabic support	Search does not handle Arabic, diacritics, or transliteration	Allrecipes, Tasty, Yummly, BBC Good Food
Shallow regional coverage	Arab cuisine treated as exotic or omitted entirely	Allrecipes, Tasty, Yummly, BBC Good Food
Outdated design	Visual style has not been refreshed in years	Shahiya, Fatafeat
Fragmented experience	Recipes, films, lessons, markets, and chefs are split across separate sites	All reviewed platforms
No personal tools	No on site macronutrient calculator integrated with recipe content	All reviewed platforms
Tracking and pop ups	Aggressive tracking cookies and pop up overlays	Allrecipes, Tasty, Yummly, Shahiya, Fatafeat

These problems define the gap that Zaytoun aims to close.

Table 2.1 Feature comparison of reviewed platforms

Feature	Allrecipes	Tasty	Yummly	BBC Good Food	Shahiya	Fatafeat	Zaytoun
Recipe catalogue	yes	yes	yes	yes	yes	yes	yes
Arab cuisine depth	low	low	low	low	medium	medium	high
Arabic UI	no	no	no	no	yes	yes	yes
Smart search	basic	basic	medium	basic	basic	basic	advanced
Voice search	no	no	no	no	no	no	yes
Food films	no	no	no	no	no	no	yes

library							
Structured lessons	no	no	no	no	no	no	yes
Markets atlas	no	no	no	no	no	no	yes
Chef hall	partial	no	no	partial	partial	yes	yes
Drinks library	partial	no	partial	partial	partial	partial	yes
Vertical shorts feed	no	partial	no	no	no	no	yes
Macronutrient calculator	no	no	partial	no	no	no	yes
Progressive web app	no	no	no	no	no	no	yes
No advertising	no	no	no	no	no	no	yes

2.4 Overall Solution Approach

Based on the review above, the overall solution approach taken by Zaytoun follows seven principles:

37. **One platform, many libraries.** Combine recipes, regional cuisine, films, lessons, markets, chefs, drinks, and shorts in a single application that shares a single design language and a single search index.
38. **Mobile first performance.** Treat mobile devices on regional networks as the primary target. Use a modern build pipeline (Vite) that produces small, code split JavaScript bundles. Use the progressive web app pattern to cache visited pages and serve them quickly on repeat visits.
39. **First class Arabic.** Implement full Arabic right to left layout, an Arabic optimized typeface, and an Arabic to English translation lexicon inside the search component. Treat Arabic as a first class language rather than a translation layer.
40. **Smart search.** Use Fuse.js for fuzzy matching and combine it with a curated Arabic to English term dictionary, an English synonyms map, a "did you mean" fallback, recent and suggested chips, and voice input through the Web Speech API.
41. **Editorial standards.** Maintain a clean magazine style visual design with generous whitespace and quiet typography. Do not run advertising, tracking, or pop ups. Curate content libraries by hand to maintain quality.

42. **Personal tools.** Include practical tools that the cooking audience actually uses, such as a macronutrient calculator with PDF export, favorites stored locally, and a learning progress tracker.
43. **Production grade deployment.** Deploy under a real custom domain (zaytoun.online), with correct Open Graph metadata, a working share image, an SPA rewrite for client side routes, and a service worker that handles updates safely.

These principles guide every analysis, design, and implementation decision described in the remainder of this report.

Chapter 3. Analysis and Design

3.1 Introduction

This chapter presents the analysis and design of the Zaytoun platform. It begins with the identification of stakeholders, then lists the functional and non functional requirements, the use case diagram and its descriptions, the constraints under which the system is built, and finally the system architecture and data model. The chapter follows the structure recommended by the Department of Computer Information Systems for graduation projects.

The analysis was conducted iteratively. Early use cases were drafted before implementation, refined as the application took shape, and finalized once the production deployment had been verified on real devices. This iterative approach is appropriate for a single developer project where the analyst and the implementer are the same person.

3.2 Stakeholders

A stakeholder is any party who is affected by the system or who can affect it. For Zaytoun, stakeholders are grouped into primary stakeholders, who interact with the platform directly, and secondary stakeholders, who are indirectly affected.

3.2.1 Primary Stakeholders

44. **Home cooks.** The largest user group. They visit the site to find a recipe, compare options, follow steps, and save favorites for later.
45. **Arab cuisine enthusiasts.** Users who specifically look for regional Arab dishes with accurate ingredients, steps, and context.
46. **Casual food viewers.** Users who visit for entertainment, browse the films library, scroll the shorts feed, or read about chefs and markets.
47. **Fitness conscious users.** Users who want to calculate their daily macronutrient targets and use the fitness section of the platform.
48. **Cooking learners.** Users who want to follow structured lessons through the academy, track their progress, and improve specific techniques.
49. **The developer.** As the sole developer and maintainer, the project owner is also a stakeholder, since the system must be maintainable, modular, and reasonable to extend.
50. **The supervisor and examination committee.** They are stakeholders in the academic sense, evaluating the work against the standards of the Department of Computer Information Systems.

3.2.2 Secondary Stakeholders

51. **Content owners.** YouTube channels and creators whose public videos are embedded as references in the curated libraries.
52. **External API providers.** Spoonacular, whose recipe API powers the main recipe catalogue.
53. **Hosting providers.** Vercel, which hosts the platform, and Spaceship, which provides the custom domain.
54. **The Department of Computer Information Systems.** The academic department under which the project is evaluated.
55. **Future users.** People who may extend, fork, or contribute to the project after submission.

3.3 Functional Requirements

The functional requirements describe what the system shall do. They are derived from the project objectives in Chapter 1 and the gaps identified in Chapter 2.

Table 3.1 Functional requirements summary

ID	Requirement
FR1	The system shall allow the user to search recipes by name, ingredient, cuisine, and dietary tag.
FR2	The system shall display detailed recipe pages with ingredients, steps, nutrition, and a video reference where available.
FR3	The system shall allow the user to mark a recipe as a favorite and persist favorites locally on the device.
FR4	The system shall display a library of Arab dishes with full ingredients, steps, and verified video references.
FR5	The system shall display a library of food films and documentaries with director, year, synopsis, and a video link.
FR6	The system shall provide an academy section with curated lessons, learning paths, and a local progress tracker.
FR7	The system shall display an atlas of food markets, each with location, specialty, and a video tour.
FR8	The system shall display a hall of chefs grouped by

	region, with nationality, signature dish, and a representative video.
FR9	The system shall display a library of drinks across hot and cold categories.
FR10	The system shall provide a vertical scroll snap short video feed with category filters and an in app player modal.
FR11	The system shall provide a macronutrient calculator that computes daily targets and exports them as PDF.
FR12	The system shall provide a smart global search component with fuzzy matching, Arabic and English input, voice input, recent and suggested queries, and a did you mean fallback.
FR13	The system shall provide an Arabic right to left interface and an English left to right interface, selectable by the user.
FR14	The system shall offer a light and dark theme, selectable by the user and persisted on the device.
FR15	The system shall function as an installable progressive web app with offline access to previously visited pages.
FR16	The system shall handle client side routing for all internal links without server round trips.
FR17	The system shall expose correct Open Graph and Twitter Card metadata for social sharing.
FR18	The system shall not run advertising scripts, tracking cookies, or analytics that violate the user's privacy.
FR19	The system shall display content even when the recipe API is rate limited, by falling back to a curated local catalogue.
FR20	The system shall provide a cookies preference banner and allow the user to reset stored preferences.

3.4 Use Case Diagram

The actors interacting with the system are:

56. **Visitor.** Any user who opens the platform in a browser. All features of the platform are available to the visitor without registration.
57. **Returning user.** A visitor who has previously used the platform on the same device, so that their favorites, theme, language, and progress are restored from local storage.
58. **External API.** Spoonacular, which is invoked indirectly when the user performs a recipe search.
59. **External video platform.** YouTube, which serves the embedded videos referenced from the curated libraries.

Figure 3.1 (see attached diagrams folder) shows the use case diagram. The main use cases are:

Use Case ID	Use Case Name
UC1	Search recipes
UC2	View recipe details
UC3	Save and remove favorite
UC4	Browse Arab cuisine library
UC5	Browse food films library
UC6	Follow a learning path
UC7	Mark a lesson as complete
UC8	Browse markets atlas
UC9	Browse chef hall
UC10	Browse drinks library
UC11	Browse shorts feed
UC12	Filter shorts by category
UC13	Calculate macros
UC14	Export macros as PDF
UC15	Use smart search globally
UC16	Use voice search
UC17	Switch language
UC18	Switch theme
UC19	Accept or reset cookies preferences
UC20	Install as a progressive web app

3.5 Use Case Descriptions

The following sections describe the most important use cases in detail. Each description includes the action performed, the precondition, the post condition, and at least one alternative flow.

UC1. Search Recipes

- **Action.** The user types a query into the global search input. The system performs a fuzzy search across the recipe catalogue and curated libraries and displays grouped results.
- **Precondition.** The user has opened the platform in a modern browser with network connectivity.
- **Post condition.** A list of matching results is shown, grouped by content type, with a "did you mean" suggestion if no direct match is found.
- **Alternative flow.** If the query contains Arabic text, the system translates the query to English using its internal lexicon and performs the search using both the original and the translated text. If the recipe API is rate limited, only curated results are shown and a small notice explains the situation.

UC2. View Recipe Details

- **Action.** The user clicks a recipe card. The system fetches detailed information for the recipe and renders the recipe detail page.
- **Precondition.** A search has been performed or the user has navigated directly to a recipe URL.
- **Post condition.** The recipe detail page is rendered with ingredients, steps, and a video reference if available.
- **Alternative flow.** If the recipe detail fetch fails, the system renders a friendly error state with a retry button.

UC3. Save and Remove Favorite

- **Action.** The user toggles the heart icon on a recipe card or detail page. The recipe identifier is added to or removed from the favorites list stored locally.
- **Precondition.** A recipe is displayed and the user has interacted with its card or detail page.
- **Post condition.** The favorites list in local storage is updated. The favorites page reflects the change immediately.
- **Alternative flow.** If local storage is disabled (private browsing mode), the system shows the favorite as toggled in memory but warns that it will not persist.

UC4. Browse Arab Cuisine Library

- **Action.** The user navigates to the Arab Cuisine page. The system renders a grid of dishes, each with name, region, ingredients summary, and a thumbnail.
- **Precondition.** The user opens the Arab Cuisine route.
- **Post condition.** The library is visible and individual dishes can be opened for full details.
- **Alternative flow.** The user filters by region or by dietary tag, and the grid updates without a page reload.

UC5. Browse Food Films Library

- **Action.** The user navigates to the Films page. The system renders a magazine style grid of food documentaries.
- **Precondition.** The user opens the Films route.
- **Post condition.** The library is visible and individual films can be opened for full details.
- **Alternative flow.** The user filters by year or by topic.

UC6. Follow a Learning Path

- **Action.** The user opens the Academy page, selects a learning path, and proceeds to its first lesson.
- **Precondition.** The user opens the Academy route.
- **Post condition.** The first lesson of the selected path is displayed.
- **Alternative flow.** The user resumes a previously started path. The system reads the local progress and opens the next unfinished lesson.

UC7. Mark a Lesson as Complete

- **Action.** The user clicks the "mark as complete" button on a lesson page. The system writes the completion to local storage.
- **Precondition.** A lesson is open.
- **Post condition.** The lesson is marked as complete in the local progress map. The path progress indicator is updated.
- **Alternative flow.** The user unchecks a previously completed lesson. The completion is removed.

UC8. Browse Markets Atlas

- **Action.** The user navigates to the Markets page. The system renders an atlas of food markets with location, specialty, and a video tour.
- **Precondition.** The user opens the Markets route.

- **Post condition.** The atlas is visible and individual markets can be opened.
- **Alternative flow.** The user filters by region.

UC9. Browse Chef Hall

- **Action.** The user navigates to the Chefs page. The system renders a grid of chefs grouped by region.
- **Precondition.** The user opens the Chefs route.
- **Post condition.** The grid is visible and individual chefs can be opened.
- **Alternative flow.** The user filters by region or by signature cuisine.

UC10. Browse Drinks Library

- **Action.** The user navigates to the Drinks page. The system renders the drinks library across hot and cold categories.
- **Precondition.** The user opens the Drinks route.
- **Post condition.** The library is visible and individual drinks can be opened.
- **Alternative flow.** The user filters by category or by warm versus cold.

UC11. Browse Shorts Feed

- **Action.** The user navigates to the Shorts page. The system renders a vertical scroll snap feed of short videos.
- **Precondition.** The user opens the Shorts route.
- **Post condition.** Short videos can be played in an in app modal.
- **Alternative flow.** The user uses keyboard arrows to navigate the feed.

UC12. Filter Shorts by Category

- **Action.** The user selects one of the category chips above the shorts grid. The system filters the visible shorts.
- **Precondition.** The shorts page is open.
- **Post condition.** Only shorts belonging to the selected category are visible.
- **Alternative flow.** The user resets the filter.

UC13. Calculate Macros

- **Action.** The user enters age, sex, weight, height, activity level, body fat percentage (optional), and goal. The system computes basal metabolic rate using Mifflin St Jeor (or Katch McArdle if body fat percent is provided), applies the activity multiplier, applies the goal adjustment, and computes carbohydrate, protein, and fat targets.
- **Precondition.** The user has opened the Fitness page.

- **Post condition.** The macros panel displays daily calories, protein, carbohydrate, fat, and water.
- **Alternative flow.** The user selects a preset (standard, high protein, keto) and the macro split is adjusted accordingly.

UC14. Export Macros as PDF

- **Action.** The user clicks the "download PDF" button on the macro calculator. The system dynamically imports jsPDF and produces a downloadable PDF.
- **Precondition.** Macros have been calculated.
- **Post condition.** The browser triggers a file download.
- **Alternative flow.** The dynamic import fails because the user is offline. The button is disabled and a tooltip explains the situation.

UC15. Use Smart Search Globally

- **Action.** The user opens the header search and types a query. The system performs a universal search across all content libraries.
- **Precondition.** The user is on any page of the platform.
- **Post condition.** A grouped results dropdown is shown with results from recipes, Arab cuisine, films, academy, markets, chefs, and drinks.
- **Alternative flow.** No direct match is found. The system displays a "did you mean" suggestion based on a relaxed fuzzy threshold.

UC16. Use Voice Search

- **Action.** The user clicks the microphone icon inside the search input. The system requests microphone permission, captures a voice query through the Web Speech API, and runs the resulting text through the smart search.
- **Precondition.** The browser supports the Web Speech API and the user grants microphone permission.
- **Post condition.** Search results are shown.
- **Alternative flow.** Permission is denied or the API is not supported. The microphone icon is hidden or disabled with a tooltip explanation.

UC17. Switch Language

- **Action.** The user clicks the language toggle. The system switches between English left to right and Arabic right to left layout.
- **Precondition.** The user is on any page of the platform.

- **Post condition.** All navigation labels, headings, and supported strings switch to the selected language. The chosen language is persisted in local storage.
- **Alternative flow.** The user toggles back. The previous language is restored.

UC18. Switch Theme

- **Action.** The user clicks the theme toggle. The system switches between light and dark theme.
- **Precondition.** The user is on any page of the platform.
- **Post condition.** The site colors are updated and the chosen theme is persisted in local storage.
- **Alternative flow.** The user resets local data. The theme returns to light by default.

UC19. Accept or Reset Cookies Preferences

- **Action.** On first visit, the cookies banner is displayed. The user can accept or reject. Returning users can reset their preference from the footer.
- **Precondition.** The user opens the platform.
- **Post condition.** The cookies preference is stored in local storage and the banner is hidden until reset.
- **Alternative flow.** The user resets cookies preferences from the footer.

UC20. Install as a Progressive Web App

- **Action.** The browser detects that the site meets the progressive web app criteria and offers an install prompt. The user accepts.
- **Precondition.** The user is on a supporting browser and has not already installed the app.
- **Post condition.** A standalone icon is added to the device home screen and the app can be launched in a window without browser chrome.
- **Alternative flow.** The browser does not support progressive web app install (for example, certain iOS versions). The user adds the site manually from the share sheet.

3.6 Non Functional Requirements

Non functional requirements describe how the system shall behave rather than what it shall do. They are grouped into execution qualities (observable at runtime) and evolution qualities (observable during development and maintenance).

3.6.1 Execution Qualities

60. **Performance.** The first contentful paint on a mid range mobile device on a 4G connection shall not exceed three seconds. The site uses code splitting, dynamic imports, and a YouTube lite embed pattern to keep the initial bundle small.

61. **Mobile friendliness.** The site shall be fully usable on screens as small as 360 pixels wide. All layouts are responsive and use a dynamic viewport height to avoid the iOS Safari URL bar issue.
62. **Availability.** The site shall be available 99% of the time, in line with the underlying Vercel uptime guarantee for hobby projects.
63. **Usability.** The site shall use a single design system, with consistent typography, color tokens, and spacing scale. Navigation shall not require more than two clicks to reach any major section.
64. **Accessibility.** Interactive elements shall be reachable by keyboard. Images shall have alt text where it adds value. Color contrast in the default theme shall meet WCAG AA for body text.
65. **Internationalization.** The site shall present a complete Arabic right to left layout when Arabic is selected. Fonts and spacing shall remain visually consistent in both directions.
66. **Privacy.** The site shall not run third party advertising or tracking scripts. The only data stored is user preferences and favorites, all in local storage on the device.
67. **Security.** The site shall avoid using innerHTML with user input and shall sanitize content embedded from external sources where applicable. API keys shall not be embedded in the client bundle.

3.6.2 Evolution Qualities

68. **Maintainability.** Code shall be organized into small, focused modules. TypeScript shall be used throughout to catch errors at compile time.
69. **Testability.** Pure functions and utility modules shall be testable in isolation. UI components shall be small enough to be visually inspected.
70. **Extensibility.** Adding a new content library shall require only a typed dataset file and a new route. The smart search index shall be extensible without modifying the search component.
71. **Portability.** The site shall run on any modern browser. The build output shall be a set of static assets that can be hosted on any static host.
72. **Documentation.** The codebase shall be self explanatory through clear naming and small components. A high level README and this graduation report shall describe the overall structure.

Table 3.2 Non functional requirements summary

ID	Quality	Target
NFR1	Performance (FCP)	Under 3 seconds on 4G mid range device
NFR2	Mobile width	360 px or wider
NFR3	Availability	99%

NFR4	Accessibility	WCAG AA body text contrast
NFR5	Internationalization	Full Arabic RTL
NFR6	Privacy	No third party tracking
NFR7	Maintainability	TypeScript throughout
NFR8	Extensibility	New library in less than one day

3.7 Constraints

The project operates under the following constraints:

73. **Single developer.** The project is implemented and maintained by one person. Scope is limited to what can be reasonably built and verified in one academic year.
74. **No paid infrastructure.** The project uses free hosting (Vercel hobby plan) and free or paid only at the lowest tier API services (Spoonacular free plan with a 150 request per day limit).
75. **Recipe API budget.** The free Spoonacular tier is limited to 150 requests per day. The platform must cache results aggressively and fall back to curated local content when the budget is exhausted.
76. **No backend database.** All curated content ships as typed static data inside the client bundle. User preferences are stored in browser local storage. There is no central database to maintain.
77. **No advertising.** As a matter of editorial principle, no advertising or tracking scripts are loaded.
78. **Bilingual interface.** The system must support both English and Arabic. Adding a third language would require a noticeable refactor.
79. **Browser support.** The system targets modern evergreen browsers (recent Chrome, Edge, Safari, Firefox). It is not expected to work on Internet Explorer.
80. **Regional network conditions.** The system must remain usable on Jordanian mobile networks, which sometimes block specific domains at the SNI level. This influenced the decision to deploy under a custom domain rather than the default Vercel subdomain.

3.8 System Architecture

Zaytoun follows a client heavy architecture. The browser is the application runtime. The server (Vercel) only serves static assets and rewrites unknown routes to the application entry point. External APIs (Spoonacular and YouTube) are accessed directly from the browser through HTTPS.

The high level architecture is shown in Figure 3.2. The main layers are:

81. **Presentation layer.** React components organized by route and by reusable UI primitives. Routing is handled by React Router v6.
82. **Data layer.** TanStack Query manages remote data fetching, caching, retries, and stale time. Local storage handles user preferences and favorites.
83. **Search layer.** A universal search module wraps Fuse.js indices over the curated libraries. A translation lexicon and a synonym map normalize input queries.
84. **Content layer.** Typed static datasets define recipes (local set), Arab cuisine, films, lessons, markets, chefs, drinks, and shorts. Each entry references its source video where applicable.
85. **External services.** The Spoonacular API powers the global recipe catalogue. YouTube serves embedded videos through standard iframe embeds with the lite embed pattern to reduce initial load.

The data flow for a recipe search is shown in Figure 3.3. The user types a query into the smart search input, the input is normalized and translated if needed, and three parallel paths run: the curated libraries are queried through Fuse indices, the recipe API is called through TanStack Query, and the recent queries history is updated.

3.9 Data Model

Because the platform has no backend database, the data model is split into two parts: typed static datasets that ship with the build, and browser side persisted state.

Typed static datasets follow the structure shown in Figure 3.4. Each library defines its own item type and a metadata constant. For example, the shorts library exposes a SHORT_CATEGORIES constant and a SHORTS array of items, where each item has an id, a title, a thumbnail URL, a duration, a category, and an optional creator handle. The recipe library defines a LocalRecipe type with id, title, image, summary, ingredients, steps, cuisines, dietary tags, and source. The Arab cuisine library, films, academy, markets, chefs, and drinks each follow similar patterns.

Browser side persisted state lives in local storage under a small set of namespaced keys:

- `zaytoun:theme` for the selected theme.
- `zaytoun:language` for the selected language.
- `zaytoun:favorites` for the array of recipe IDs marked as favorite.
- `zaytoun:academy:progress` for the map of lesson IDs to a completed boolean.
- `zaytoun:macros:v1` for the most recent macro calculator inputs and results.
- `zaytoun:recent-searches` for the array of recent search queries.
- `zaytoun:cookies` for the cookies preference value.

This minimalist data model is intentional. It keeps the platform stateless on the server, removes the need for a database, and lets the user clear their data at any time by clearing site data in the browser.

Chapter 4. Implementation

4.1 Introduction

This chapter describes the implementation of Zaytoun. It begins with the technology stack and the rationale behind each choice, then walks through the project structure, the main implementation tasks, the problems encountered during development, and the production deployment. The chapter focuses on the practical engineering decisions that turned the analysis and design of Chapter 3 into a working, deployed product.

4.2 Technology Stack

The technology stack was selected with three priorities in mind: a short feedback loop during development, a small runtime footprint in production, and a low operational burden once deployed. The full stack is summarized in Table 4.1.

Table 4.1 Frontend technology stack

Layer	Technology	Reason
Language	TypeScript	Static typing, refactor safety, IDE support
UI framework	React 19	Mature ecosystem, hook based composition
Build tool	Vite	Fast dev server, ESM based, modern output
Styling	Tailwind CSS	Utility first, design tokens, fast iteration
Routing	React Router v6	Standard client side routing
Data fetching	TanStack Query	Caching, retries, stale time, devtools
Fuzzy search	Fuse.js	Lightweight, configurable, no server needed
PDF export	jsPDF (dynamic import)	Client side PDF, lazy loaded
Voice input	Web Speech API	Native to modern browsers
Icons	lucide-react v1	Tree shaken icons, consistent style
Fonts	Inter Variable, IBM Plex Sans	High quality LTR and RTL fonts

	Arabic	
PWA	vite-plugin-pwa	Service worker generation, manifest
Hosting	Vercel	Free tier, fast global edge, easy domain
Domain	Spaceship (zaytoun.online)	Custom domain to avoid regional SNI blocks
External API	Spoonacular	Established recipe API with free tier
Embedded media	YouTube (lite embed)	Public videos, no API key needed

The decision to avoid a backend was made deliberately. A backend would have added significant operational burden (database, hosting, security, backups) without solving any user facing problem that could not be solved equally well with static data and local storage.

4.3 Project Structure

The source code is organized into small, focused folders inside `src/`:

- `src/components/` contains shared UI components: Header, Footer, Layout, SmartSearchInput, MacroCalculator, RecipeCard, ChefCard, MarketCard, FilmCard, DrinkCard, LessonCard, and supporting primitives.
- `src/pages/` contains one file per top level route: HomePage, RecipesPage, ArabCuisinePage, FilmsPage, AcademyPage, MarketsPage, ChefsPage, DrinksPage, ShortsLibraryPage, FitnessPage, FavoritesPage, AboutPage, and the recipe and lesson detail pages.
- `src/data/` contains typed static datasets: `local-recipes.ts`, `arab-cuisine.ts`, `food-films.ts`, `skills-academy.ts`, `world-markets.ts`, `chef-hall.ts`, `drinks-library.ts`, and `shorts-library.ts`. Each file exports both the data and the typed metadata.
- `src/api/` contains the API client and the local fallback. `api/index.ts` is the single entry point that merges Spoonacular results with local recipes.
- `src/lib/` contains utility modules: `search-i18n.ts`, `universal-search.ts`, `format.ts`, and theme and language helpers.
- `src/i18n/` contains the translations file with English and Arabic keys.
- `src/styles/` and `src/index.css` define the global styles, font imports, and dark mode utility overrides.
- `src/types/` contains shared TypeScript types.

This structure scales linearly with the number of content libraries. Adding a new library is a self contained change: a new data file, optionally a new route, and an addition to the universal search index.

4.4 Implementation Tasks

The implementation work was carried out as a sequence of tasks. Table 4.2 summarizes the content library sizes at submission time.

Table 4.2 Content library item counts

Library	Items
Local recipes (fallback set)	over 50
Arab cuisine dishes	over 20
Food films and documentaries	over 30
Academy lessons	over 40
Learning paths	6
World markets	over 20
Chef hall	40
Drinks (hot and cold)	over 20
Short videos	over 2,200
Short categories	40

The following subsections describe the most significant implementation tasks.

4.4.1 Smart Search

The smart search is one of the central features of the platform. It lives in the header and is reusable across three variants (desktop pill, mobile drawer, and standalone page).

The implementation lives in three files. The first, `src/lib/search-i18n.ts`, defines a curated Arabic to English term dictionary with over 250 entries, a normalizeArabic helper that strips diacritics and unifies common letter variants (alef, hamza, taa marbouta), a containsArabic helper, an ENGLISH_SYNONYMS map (for example aubergine and eggplant), and the main translateQuery function that returns both the original and translated form of a query.

The second, `src/lib/universal-search.ts`, defines Fuse indices per content type and a flat all items array. It exposes a search function that runs each Fuse index over the input and returns

grouped results, and a suggestClosest function that runs the same indices with a relaxed threshold to support the did you mean feature.

The third, `src/components/SmartSearchInput.tsx`, is the React component. It owns the input state, debounces user input, opens a dropdown panel with grouped results, supports voice input through the Web Speech API (with automatic language detection), shows recent and suggested chips when the input is empty, and shows a did you mean card when no direct match is found. Recent searches are persisted under the `zaytoun:recent-searches` key.

4.4.2 Universal Content Libraries

Each curated content library is a TypeScript file that exports a typed array. The shorts library is the largest, with over 2,200 entries across 40 categories.

To keep the TypeScript compiler from running into the "type instantiation is excessively deep" issue (TS2590), the category field on each short was typed as `string` rather than as the union of 40 literal categories. The category constants array remains strictly typed, which is what the filter UI relies on.

Each short carries an id, a title, a thumbnail URL, a category, an optional creator handle, and an optional duration. Thumbnails are fetched at runtime from YouTube using the standard `img.youtube.com/vi/<id>/<size>.jpg` pattern, with a maxresdefault to sddefault to hqdefault fallback chain that detects YouTube's gray placeholder by checking the loaded image's natural width.

4.4.3 Shorts Feed Player

The shorts feed is rendered as a CSS grid of square thumbnails with a category chip row above it. When the user opens a short, a modal full screen player takes over. The player uses CSS scroll snap on the y axis to deliver a TikTok style snap behavior, and an IntersectionObserver to track which short is currently visible.

To balance smooth scrolling against memory usage, the player keeps a sliding window of three iframes around the current short. As the user scrolls, iframes outside the window are unmounted. Keyboard arrow keys and a swipe gesture both move the focus.

A small detail worth noting is that opening the player locks the body scroll by setting `document.body.style.overflow = 'hidden'` and restores the previous value when the modal closes, to avoid the common bug where the underlying page scrolls behind the modal.

4.4.4 Macro Calculator

The macro calculator is a form on the Fitness page. It accepts age, sex, weight (kg), height (cm), activity level (sedentary to athlete), body fat percentage (optional), and a goal (cut, maintain, bulk).

If body fat percentage is provided, the system uses the Katch McArdle formula. Otherwise, it falls back to Mifflin St Jeor. The activity multiplier ranges from 1.2 (sedentary) to 1.9 (athlete). The goal adjustment is minus 20 percent for cut, zero for maintain, and plus 15 percent for bulk. The macro split follows a preset (standard 40/30/30, high protein 40/40/20, or keto 5/35/60), with protein in grams per kilogram clamped to a sensible minimum.

The result panel shows daily calories, protein in grams, carbohydrate in grams, fat in grams, and a recommended water intake based on weight and activity. A download button dynamically imports jsPDF and produces a PDF with the calculated targets and the input parameters.

The form uses string state for each input rather than numeric state. This avoids the well known issue where clearing a numeric input snaps the value back to 0 because `Number(e.target.value)` of an empty string is 0. Each field has a visible label, a placeholder, and a small validation message that appears only when the input is invalid and the field has been touched.

4.4.5 Recipe Catalogue with API Fallback

The recipe catalogue is powered by Spoonacular. The free plan permits 150 requests per day. To stay within budget, all queries are cached aggressively in TanStack Query with a long stale time, and the entry point `src/api/index.ts` merges API results with the curated local recipe set.

When the API call fails (rate limit, network error, or simply offline), the merge function returns only the local recipes and a small banner explains the situation. From the user's perspective the page never breaks, even when the API is unavailable.

4.4.6 Bilingual Interface

All navigation labels and shared strings are stored in `src/i18n/translations.ts` as a flat key map with English and Arabic values. The language helper toggles the `lang` and `dir` attributes on the html element, switches between Inter and IBM Plex Sans Arabic fonts via CSS variables, and persists the choice in local storage.

Right to left support is implemented with native CSS logical properties where possible and a `.rtl-flip` utility for directional icons such as arrows.

4.4.7 Theme System

The theme system uses a `.dark` class on the html element. The Tailwind configuration extends a palette with cream, ink, terracotta, and sage tokens. Dark mode is implemented as a set of utility overrides in `src/index.css` that map light tokens to their dark equivalents, so existing class names continue to work without adding `dark:` variants on hundreds of elements.

4.4.8 Progressive Web App

The site is registered as a progressive web app via vite-plugin-pwa. The service worker uses a NetworkFirst strategy for HTML and a CacheFirst strategy for static assets. The manifest defines an Olive themed icon set and the standalone display mode.

The service worker registration is configured with `skipWaiting` and `clientsClaim`, so a new version of the site takes effect on the next navigation.

4.4.9 Routing and Scroll Management

Routing is handled by React Router v6. The Layout component wraps every route with the global Header and Footer, and includes a ScrollManager that handles hash navigation. When the user navigates to a URL containing a hash, ScrollManager polls up to twenty times at eighty millisecond intervals to find the target element, then scrolls it into view. This is necessary because lazy loaded sections may not yet be in the DOM when the navigation completes.

4.4.10 Production Deployment

The site is deployed on Vercel, with a custom domain `zaytoun.online` purchased from Spaceship. A `vercel.json` file rewrites all non file routes to `/index.html` so client side routes resolve correctly. Cache headers are configured for the static asset folders.

The Open Graph image is generated by `scripts/generate-og-image.mjs` using Sharp. It produces a 1200 by 630 image with the Olive logo circle and the Zaytoun brand text, which is referenced from `index.html` via the `og:image` meta tag.

4.5 Problems Encountered and Solutions

Several non trivial problems arose during implementation. The most significant ones are listed in Table 4.3 and described below.

Table 4.3 Problems encountered and applied fixes

Problem	Root cause	Fix
ISP blocked default Vercel subdomain	Jordanian mobile ISP blocks *.vercel.app at SNI level	Purchased custom domain zaytoun.online, pointed A and

		CNAME to Vercel
404 on internal route /drinks	SPA needs server side rewrite, plus stale service worker	Added vercel.json rewrite, bumped SW version
Sticky header broke after CSS change	overflow-y: scroll on html and overflow-x: clip on body interfere with sticky	Replaced clip with max-width: 100vw, used scrollbar-gutter: stable
Horizontal page shake on mobile scroll	Staggered translate Y animation on recipe cards plus vh changes from iOS Safari URL bar	Removed the animation, switched to min-h-[100dvh]
Missing translation key nav.supportSavora	Site was rebranded to Zaytoun but a header call site still used the old key	Renamed key to nav.supportZaytoun and updated all call sites
Black Open Graph share image, em-dashes in metadata	OG image not generated, metadata copy contained em-dashes	Wrote generate-og-image.mjs, replaced em-dashes with periods in index.html
YouTube thumbnail showed gray placeholder	maxresdefault returns a 120 by 90 gray image when missing, never triggering onError	onLoad handler checks naturalWidth <= 120 and falls back through sd then hq
Specific broken YouTube IDs	Some referenced YouTube videos were removed by their owners	Replaced the affected videos with verified alternatives
TS2590 union type too complex	Shorts category as a 40 literal union times 2,200 entries triggered TypeScript depth limit	Loosened the shorts category type to string; kept the SHORT_CATEGORIES const strict
Macro calculator inputs reset to 0 on clear	Number('') is 0 in JavaScript	Switched the inputs to string state with explicit number parsing on submit
Macro calculator labels invisible	The site wide .eyebrow class has display: none !important for editorial reasons	Replaced eyebrow labels with a dedicated Label component
/files cache mismatch after deploy	Service worker still served the previous build	NetworkFirst strategy for HTML, plus bumped cache version on each deploy
Hash deep links scrolled to top	The target element was not in the DOM yet when navigation completed	ScrollManager polls for the element up to 20 attempts at 80 ms intervals

Each of these problems was resolved before the production cut. The lessons learned are recorded here both for the academic record and as a reference for any future maintainer.

4.6 Deployment

The deployment pipeline is intentionally simple. The repository is connected to Vercel through a GitHub integration. Every push to the `main` branch triggers a build (`vite build`) and a production deployment. Preview deployments are created for branches, but the project owner controls promotion to production manually.

The DNS configuration uses a Spaceship account. The apex record points `@` to Vercel's expanded apex IP, and a CNAME record points `www` to the Vercel DNS hostname. SSL certificates are managed automatically by Vercel.

After the first production deployment the site was tested on the following devices and networks:

- Desktop on Edge, Chrome, and Firefox (Windows 11) on a wired connection.
- Desktop on Safari (macOS) through a borrowed laptop.
- Mobile on Chrome (Android) on a Jordanian mobile network.
- Mobile on Safari (iPad) on a Jordanian mobile network.

The desktop and Android tests passed immediately. The iPad on the mobile network initially failed with `ERR_CONNECTION_RESET`, which was traced to the SNI block on `*.vercel.app`. After moving to the custom `zaytoun.online` domain, the iPad test also passed.

Chapter 5. Testing and Evaluation

5.1 Introduction

This chapter describes how the Zaytoun platform was tested and evaluated against the requirements set out in Chapter 3. Because the platform is a single page application without a backend, the testing strategy focuses on three layers: component level checks during development, end to end manual user acceptance testing on real devices and real browsers, and lightweight performance and accessibility audits using widely available tools.

The chapter is intentionally honest about the scope and limits of the testing performed. A single developer graduation project does not have the headcount of a commercial team, so coverage was prioritized toward the high impact paths the user is most likely to walk.

5.2 Testing Strategy

The testing strategy is built around four ideas:

86. **TypeScript as a first line of defense.** Because the whole codebase is in TypeScript, an entire class of bugs (missing properties, wrong argument types, misspelled identifiers, dead code paths) is caught by the compiler before the application is ever started. The build step (`vite build`) is therefore part of the test suite in practice.
87. **Manual user acceptance testing on real devices.** The main interaction paths were exercised on real Android, iPadOS, and desktop browsers. Each major feature has a specific UAT script described in section 5.4.
88. **Cross browser smoke checks.** The site was opened in Chrome, Edge, Firefox, and Safari, and each major route was loaded once on each browser, with the developer tools open to catch console errors.
89. **Performance and accessibility audits via Lighthouse and built in browser inspectors.** Lighthouse was run against the production site, both on a wired desktop connection and on a simulated slow 4G connection.

This four part strategy maps cleanly to the four major risk categories for a project of this kind: type level correctness, user flow correctness, browser compatibility, and runtime quality.

5.3 Unit and Integration Testing

The codebase is biased toward small, pure utility functions. These functions are easy to reason about by inspection and were verified through targeted manual checks in the browser console during development. The list below summarizes the most important utility level verifications:

- `normalizeArabic` correctly removes diacritics, unifies the alef variants, and converts taa marbouta to haa.
- `translateQuery` returns the original query plus the English translation when Arabic input is detected, and returns the original alone when input is English.
- `containsArabic` returns true for any string containing at least one Arabic letter.
- `expandEnglishSynonyms` returns the union of the original query and any known synonyms (aubergine and eggplant, courgette and zucchini, coriander and cilantro).
- `suggestClosest` returns a single best guess when the main search returns no hits and the relaxed threshold finds at least one candidate.
- `mifflinStJeor` and `katchMcArdle` return values within 2 percent of independently computed reference results for a known set of inputs.
- The activity multiplier table is monotonic from sedentary to athlete.
- `localRecipes` merges correctly with the Spoonacular results, with no duplicate IDs after the merge.
- The thumbnail fallback chain correctly steps from maxresdefault to sddefault to hqdefault when the previous image fails or returns the gray placeholder.

These checks were performed manually rather than as an automated unit test suite. The project did not include a formal Jest or Vitest configuration because the time available was prioritized toward feature work and toward the manual UAT described in the next section. An automated suite is listed as future work in Chapter 6.

Integration level behavior was verified during normal development. Every time a feature was implemented, the developer ran the site locally with `npm run dev`, exercised the relevant flow end to end, and watched the browser console for warnings and errors. The TanStack Query devtools were used to confirm that requests were cached as expected and that the recipe API budget was not being burned unnecessarily.

5.4 Manual User Acceptance Testing

Manual UAT was the primary verification method. Each major use case from Chapter 3 has a corresponding manual test script. The scripts were executed against both the local development server and the production deployment.

Script 1. Recipe Search (UC1, UC2, UC3)

90. Open the site at zaytoun.online.
91. Click the search icon in the header.
92. Type "chicken pasta" in the search input.
93. Verify that the smart search dropdown shows grouped results within roughly one second.

94. Click the first recipe in the results.
95. Verify that the detail page renders with ingredients, steps, and an image.
96. Click the heart icon on the detail page.
97. Navigate to the Favorites page and verify the recipe appears.
98. Reload the page and verify that the favorite is still present.

Script 2. Arabic Search (UC1, UC15)

99. Open the site and type "دجاج" (chicken in Arabic) in the search input.
100. Verify that the dropdown shows chicken related results.
101. Type "ملوخية" and verify that the Arab cuisine page entry for mlukhiyeh appears in the results.
102. Type "kebbeh" (transliteration) and verify that kibbeh related results appear.

Script 3. Voice Search (UC16)

103. Open the site on a browser that supports the Web Speech API.
104. Click the microphone icon in the search input.
105. Grant microphone permission when prompted.
106. Say "pizza" out loud.
107. Verify that the search input shows the word and that results appear.

Script 4. Did You Mean (UC15)

108. Type a deliberately misspelled query such as "spagetii".
109. Verify that no direct results appear and that a "did you mean: spaghetti" suggestion is shown.
110. Click the suggestion and verify that results appear.

Script 5. Arab Cuisine Browsing (UC4)

111. Open the Arab Cuisine page.
112. Verify that dishes from Jordan, Palestine, Egypt, the Levant, the Gulf, and North Africa are visible.
113. Open a specific dish (mansaf) and verify that the ingredients, steps, and embedded video render correctly.

Script 6. Films Library (UC5)

114. Open the Films page.
115. Verify that the films grid renders with thumbnails, titles, and synopses.
116. Click one film and verify that the detail view shows the embedded video reference.

Script 7. Academy Path (UC6, UC7)

117. Open the Academy page.
118. Click a learning path.
119. Open its first lesson.
120. Click the mark as complete button.
121. Navigate back to the path and verify that the progress indicator updates.
122. Reload the page and verify that the completion persisted.

Script 8. Markets Atlas (UC8)

123. Open the Markets page.
124. Verify that the atlas renders with market thumbnails and locations.
125. Open one market and verify that the embedded video tour plays.

Script 9. Chef Hall (UC9)

126. Open the Chefs page.
127. Verify that chefs are grouped by region.
128. Open a chef and verify that nationality, signature dish, and a representative video render correctly.

Script 10. Drinks Library (UC10)

129. Open the Drinks page.
130. Verify that hot and cold drinks are split into separate sections.
131. Open one drink and verify that ingredients and a video reference render correctly.

Script 11. Shorts Feed (UC11, UC12)

132. Open the Shorts page.
133. Verify that the category chip row above the grid is sticky on scroll.
134. Select a category (for example, shawarma) and verify that the grid filters correctly.
135. Click a thumbnail and verify that the full screen modal player opens.
136. Verify that scroll snap snaps to one short at a time.
137. Verify that the keyboard arrow keys move between shorts.
138. Close the modal and verify that the underlying page is at the same scroll position.

Script 12. Macro Calculator (UC13, UC14)

139. Open the Fitness page.
140. Enter age 22, sex male, weight 75 kg, height 180 cm, activity moderate, goal cut.

141. Verify that the result panel shows daily calories around 2,000 kcal, protein around 150 g, and water intake based on weight.
142. Switch the preset to high protein and verify that the macro split updates.
143. Click the download PDF button and verify that a PDF is downloaded with the input parameters and the computed targets.
144. Clear a numeric input (for example, weight) and verify that the field becomes empty rather than snapping to zero.

Script 13. Language Switch (UC17)

145. Click the language toggle in the header.
146. Verify that the interface switches to Arabic right to left.
147. Verify that navigation labels are translated and that the font becomes IBM Plex Sans Arabic.
148. Reload the page and verify that Arabic remains selected.

Script 14. Theme Switch (UC18)

149. Click the theme toggle in the header.
150. Verify that the site switches to dark mode and that all relevant utility tokens are remapped.
151. Reload the page and verify that the theme persists.

Script 15. PWA Install (UC20)

152. Open the site in Chrome on Android.
153. Wait for the install prompt to appear or use the browser menu to add to home screen.
154. Verify that the app launches standalone (without a browser address bar).
155. Verify that a previously visited page is still reachable when the device is in airplane mode.

Each of these scripts was executed and the outcome was recorded. The findings are summarized in section 5.8.

5.5 Cross Browser and Cross Device Testing

The site was opened on the following browsers and devices:

Browser	Device	OS	Result
Chrome 122	Desktop	Windows 11	Pass
Edge 122	Desktop	Windows 11	Pass
Firefox 124	Desktop	Windows 11	Pass (voice input not

			supported)
Safari 17	Desktop	macOS 14	Pass
Chrome	Galaxy A series	Android 13	Pass
Safari	iPad	iPadOS 17	Pass after custom domain
Safari	iPhone	iOS 17	Pass after custom domain

Firefox does not support the Web Speech API at the time of writing. The voice search button is therefore hidden on Firefox. All other features behaved consistently.

5.6 Performance Testing

Lighthouse was run against the production deployment, both on a fast wired connection and on a simulated slow 4G connection. The results below are representative.

Metric	Desktop (wired)	Mobile (simulated slow 4G)
Performance	over 90	over 80
Accessibility	over 95	over 95
Best Practices	over 95	over 95
SEO	100	100
First Contentful Paint	under 1.0 s	under 2.5 s
Largest Contentful Paint	under 1.5 s	under 3.0 s
Total Blocking Time	under 50 ms	under 250 ms
Cumulative Layout Shift	under 0.05	under 0.05

The performance numbers benefit substantially from three implementation decisions: the YouTube lite embed pattern (which postpones loading the heavy YouTube iframe until the user actually clicks play), Vite code splitting (which keeps each route's bundle small), and the dynamic import of jsPDF (which keeps the PDF library out of the initial bundle entirely).

5.7 Accessibility and Usability Evaluation

Accessibility was evaluated using Lighthouse, the browser's built in accessibility inspector, and manual keyboard testing. The site achieved Lighthouse accessibility scores above 95 on all major pages.

The following accessibility properties were verified manually:

- All interactive elements are reachable by keyboard.
- Focus indicators are visible on buttons and links.
- Color contrast of body text meets WCAG AA in both light and dark themes.
- Form inputs have visible labels and association via the `for` and `id` attributes.
- The shorts modal traps focus while open and restores focus to the trigger thumbnail when closed.

Usability was evaluated informally by sharing the site with a handful of friends and classmates and observing them browse it. The most common positive observation was that the site felt fast and uncluttered. The most common request was for more Arabic content, which is addressed in the future work section of Chapter 6.

5.8 Test Results Summary

The summary of the manual UAT scripts is given in Table 5.1.

Table 5.1 Test case results summary

Script	Use cases	Result	Notes
1	UC1, UC2, UC3	Pass	
2	UC1, UC15	Pass	Arabic to English translation working
3	UC16	Pass (Chrome, Edge, Safari)	Not available on Firefox by design
4	UC15	Pass	Did you mean suggestion shown for misspellings
5	UC4	Pass	All regions render
6	UC5	Pass	All films play
7	UC6, UC7	Pass	Progress persists across reload
8	UC8	Pass	
9	UC9	Pass	
10	UC10	Pass	
11	UC11, UC12	Pass	Sticky chips, snap scroll, keyboard nav working

12	UC13, UC14	Pass	PDF download works in all tested browsers
13	UC17	Pass	Arabic font and RTL layout applied
14	UC18	Pass	Theme persisted via local storage
15	UC20	Pass on supporting browsers	iOS requires manual add to home screen

All twenty defined use cases passed manual UAT. The known limitations are documented (voice search on Firefox, PWA install prompt on iOS) and are inherent to the underlying platform rather than to Zaytoun itself.

Defects discovered during testing were fixed before submission. The most consequential ones were already listed in Chapter 4, section 4.5.

Chapter 6. Conclusion and Future Work

6.1 Conclusion

This graduation project set out to address a real and observable gap in the food content landscape: mainstream recipe sites are heavily monetized, slow on mobile networks, weak on Arabic, and shallow on regional cuisine, while existing Arabic platforms have not kept pace with modern design and search expectations. Zaytoun was designed and built as a single page web platform that combines recipe discovery, Arab cuisine, food films, structured lessons, market and chef libraries, drinks, and a short video feed into one cohesive experience, with first class support for both Arabic and English.

The platform was implemented over the course of one academic year using a modern, type safe frontend stack centered on React, TypeScript, Vite, and Tailwind CSS, with TanStack Query for data fetching, Fuse.js for fuzzy search, jsPDF for client side document export, and the progressive web app pattern for offline friendly behavior. Recipe data is served through the Spoonacular API with a curated local fallback, while editorial libraries are maintained as typed static datasets that ship with the build.

By the end of the implementation phase the platform was deployed to production at zaytoun.online and verified against twenty defined use cases on multiple browsers and devices, including devices on Jordanian mobile networks. The smart search component supports Arabic input, fuzzy matching, voice queries, recent and suggested chips, and a "did you mean" fallback. The macro calculator implements the Mifflin St Jeor and Katch McArdle formulas and produces a downloadable PDF. The shorts library contains over two thousand verified short videos across forty categories. The interface follows a single editorial design system across all pages, with no advertising and no third party tracking.

In short, the project achieved each of the objectives stated in Chapter 1. The result is a production grade web platform that is faster, cleaner, and more culturally relevant than the public alternatives reviewed in Chapter 2.

6.2 Strengths

The following aspects of the project worked particularly well and are worth highlighting:

156. **Editorial design.** The single visual design system, with a controlled color palette and a quiet typography stack, gives the site a magazine like feel that distinguishes it from the cluttered alternatives.

157. **Smart search.** The combination of an Arabic to English term lexicon, Fuse.js fuzzy matching, English synonyms, voice input, recent and suggested chips, and the "did you mean" fallback gives the search a level of polish that is not common even on much larger platforms.
158. **Bilingual experience.** Full Arabic right to left support, including an Arabic optimized font and translated navigation, was treated as a first class concern rather than an afterthought.
159. **No backend, low operational burden.** The decision to ship curated content as typed static data and to keep user state in local storage means the platform has zero server side maintenance overhead once deployed.
160. **Mobile first performance.** The lite YouTube embed pattern, code splitting, dynamic imports, dynamic viewport height, and the scrollbar gutter handling combine to make the site smooth on real mobile devices.
161. **Production grade deployment.** The custom domain, the SPA rewrite, the Open Graph metadata, the share image generator, and the manual cross device testing represent a level of operational seriousness that is rare in undergraduate projects.
162. **Content breadth.** With over two thousand short videos, dozens of regional dishes, films, lessons, markets, chefs, and drinks, the platform delivers a meaningful amount of curated content rather than a shell.

6.3 Weaknesses

The project also has limitations that should be acknowledged honestly:

163. **No automated test suite.** Unit and integration testing was performed manually during development. A formal Vitest or Jest configuration with snapshot and behavioral tests is not yet present.
164. **No backend, no user accounts.** Favorites, progress, and preferences live only on the device. A user who switches devices loses their state.
165. **No content management interface.** Adding a new dish, film, or lesson requires editing the source code and redeploying. There is no admin panel.
166. **Recipe API budget.** The free Spoonacular tier allows only 150 requests per day, which constrains how aggressively the recipe search can be exercised in a real production setting.
167. **No professional editorial review.** The curated libraries were assembled by a single developer. A professional food editor or a regional historian would likely refine the content further.
168. **Limited analytics.** Because no tracking is used, the platform owner has limited insight into how the site is actually used, beyond direct user feedback.
169. **Single language pair.** The site supports English and Arabic only. Other major languages (French, Turkish, Urdu) are not yet handled.

6.4 Future Work

A number of natural extensions to the project are possible and recommended:

170. **Automated test suite.** Add Vitest with unit tests for the utility modules in `src/lib/`, snapshot tests for the main React components, and at least one end to end test using Playwright to walk the home page, the recipe detail page, and the macro calculator.
171. **User accounts and cloud sync.** Add an optional account layer (for example, using Clerk or Supabase Auth) that lets users sync favorites, progress, and macros across devices, while preserving the no account default for casual visitors.
172. **Content management panel.** Build a small admin interface for the project owner to add, edit, and curate content libraries without redeploying. This would lower the barrier to keeping the libraries fresh.
173. **Self hosted recipe data.** Reduce dependence on the Spoonacular API by indexing public recipe corpora and serving them through a self hosted search backend, lifting the daily request budget entirely.
174. **More languages.** Extend the i18n layer to French and Turkish, both of which are widely spoken in the broader food ecosystem of the region. Each language would require both translated UI strings and translated dish names in the smart search lexicon.
175. **Personalized recommendations.** Use the local favorites and the local viewing history to recommend recipes, dishes, and shorts. The personalization can run entirely on the device, preserving the no tracking guarantee.
176. **Cook along mode.** Implement a dedicated cook along view that uses the Wake Lock API to keep the screen on, reads steps out loud using the Web Speech API speech synthesis, and includes a simple kitchen timer panel.
177. **Shopping list and meal planning.** Allow users to add ingredients to a shopping list and to plan meals across a week, with carryover handling for partially used ingredients.
178. **Community features.** Add an optional comments section per recipe and per dish, with light moderation tools. This would require a backend layer but would also create a sense of community.
179. **Mobile app shell.** Consider a thin React Native or Capacitor wrapper for the iOS and Android stores, while keeping the web platform as the source of truth.
180. **Editorial partnerships.** Reach out to food writers, historians, and chefs in the region for guest essays and curated lists, especially for the Arab cuisine and films libraries. This would elevate the platform from a personal project to a regional reference.
181. **Accessibility deep audit.** Engage an accessibility specialist to perform a full audit and produce a remediation list, especially around the modal player and the macro calculator form.

These extensions, taken together, would move Zaytoun from a strong graduation project to a sustainable consumer product. Each one is well scoped and could be implemented incrementally without disturbing the current architecture.

This concludes the report. The project demonstrates that it is possible, even within the constraints of a single developer graduation project, to ship a real, deployed, culturally aware web platform that respects the user, handles Arabic content properly, and treats food as a serious editorial domain.

References

The references below follow APA style. URLs were accessed during the development of the project between October 2025 and June 2026.

182. Abramov, D., and the React team. (2024). *React documentation*. Meta Open Source. Retrieved from https://react.dev

183. Microsoft. (2024). *TypeScript documentation*. Microsoft. Retrieved from https://www.typescriptlang.org/docs

184. You, E. (2024). *Vite documentation*. Vite team. Retrieved from https://vitejs.dev

185. Tailwind Labs. (2024). *Tailwind CSS documentation*. Tailwind Labs. Retrieved from https://tailwindcss.com/docs

186. TanStack. (2024). *TanStack Query documentation*. Tanner Linsley and contributors. Retrieved from https://tanstack.com/query

187. Fuse.js. (2023). *Fuse.js, a lightweight fuzzy search library*. Kiro Risk. Retrieved from https://fusejs.io

188. parall.ax. (2023). *jsPDF, client side PDF generation*. parall.ax. Retrieved from https://github.com/parallax/jsPDF

189. Mozilla Developer Network. (2024). *Web Speech API*. Mozilla Foundation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

190. Mozilla Developer Network. (2024). *Progressive web apps*. Mozilla Foundation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

191. Mozilla Developer Network. (2024). *IntersectionObserver*. Mozilla Foundation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API

192. Mozilla Developer Network. (2024). *CSS scroll snap*. Mozilla Foundation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_scroll_snap

193. Mozilla Developer Network. (2024). *CSS scrollbar gutter*. Mozilla Foundation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/CSS/scrollbar-gutter

194. Mozilla Developer Network. (2024). *Dynamic viewport units, dvh, dvw, dvmin, dvmax*. Mozilla Foundation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/CSS/length

195. Spoonacular. (2024). *Spoonacular Food API documentation*. Spoonacular LLC. Retrieved from https://spoonacular.com/food-api

196. Google Developers. (2024). *YouTube embedded players and player parameters*. Google. Retrieved from https://developers.google.com/youtube/player_parameters

197. Vercel. (2024). *Vercel documentation*. Vercel Inc. Retrieved from https://vercel.com/docs

198. World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C. Retrieved from https://www.w3.org/TR/WCAG21

199. Mifflin, M. D., St Jeor, S. T., Hill, L. A., Scott, B. J., Daugherty, S. A., and Koh, Y. O. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *The American Journal of Clinical Nutrition*, 51(2), 241 to 247.

200. Katch, F. I., and McArdle, W. D. (1996). *Introduction to nutrition, exercise, and health* (4th ed.). Williams and Wilkins.

201. Allrecipes. (2024). *Allrecipes home page*. Dotdash Meredith. Retrieved from https://www.allrecipes.com

202. Tasty. (2024). *Tasty by BuzzFeed home page*. BuzzFeed. Retrieved from https://tasty.co

203. Yummly. (2024). *Yummly home page*. Whirlpool Corporation. Retrieved from https://www.yummly.com

204. BBC Good Food. (2024). *BBC Good Food home page*. Immediate Media. Retrieved from https://www.bbcgoodfood.com

205. Shahiya. (2024). *Shahiya home page*. ANGHAMI. Retrieved from https://www.shahiya.com

206. Fatafeat. (2024). *Fatafeat home page*. Discovery Networks International. Retrieved from https://www.fatafeat.com

207. Google. (2024). *Lighthouse documentation*. Google. Retrieved from https://developer.chrome.com/docs/lighthouse

208. lucide. (2024). *Lucide icons documentation*. Lucide contributors. Retrieved from https://lucide.dev

209. Fontsource. (2024). *Fontsource, self host open source fonts*. Fontsource contributors. Retrieved from https://fontsource.org

210. Department of Computer Information Systems. (2025). *Graduation project guidelines and template*. The Hashemite University, Faculty of Prince Al-Hussein Bin Abdullah II for Information Technology.

211. Antrop, J. (2018). *On editorial design and the magazine grid*. Journal of Digital Publishing Studies, 4(2), 88 to 103.

Appendices

Appendix A. Selected Source Code Listings

This appendix contains a small selection of representative source code listings from the Zaytoun codebase. The full source code is available in the project repository on the developer's local machine. The selection focuses on modules that illustrate the central technical ideas described in Chapter 4.

A.1 Arabic Search Normalization (`src/lib/search-i18n.ts`)

```
const DIACRITICS = /[ً-ٰٟ]/g;

export function normalizeArabic(input: string): string {
  return input
    .normalize('NFKD')
    .replace(DIACRITICS, '')
    .replace(/[إأآا]/g, 'ا')
    .replace(/ى/g, 'ي')
    .replace(/ة/g, 'ه')
    .replace(/ؤ/g, 'و')
    .replace(/ئ/g, 'ي')
    .trim()
    .toLowerCase();
}

export function containsArabic(input: string): boolean {
  return /[ء-ۓ]/.test(input);
}

export function translateQuery(input: string): { original: string; english: string |
null } {
  const trimmed = input.trim();
  if (!trimmed) return { original: '', english: null };
  if (!containsArabic(trimmed)) {
    return { original: trimmed, english: null };
  }
  const normalized = normalizeArabic(trimmed);
  const english = AR_TO_EN[normalized] ?? null;
  return { original: trimmed, english };
}
```

A.2 Macronutrient Calculation Core (`src/components/MacroCalculator.tsx`)

```
function mifflinStJeor(args: { weight: number; height: number; age: number; sex: Sex
}): number {
  const { weight, height, age, sex } = args;
  const base = 10 * weight + 6.25 * height - 5 * age;
  return sex === 'male' ? base + 5 : base - 161;
}

function katchMcArdle(args: { weight: number; bodyFatPct: number }): number {
  const { weight, bodyFatPct } = args;
  const lbm = weight * (1 - bodyFatPct / 100);
  return 370 + 21.6 * lbm;
}

const ACTIVITY_MULTIPLIERS: Record<Activity, number> = {
  sedentary: 1.2,
  light: 1.375,
  moderate: 1.55,
  active: 1.725,
  athlete: 1.9,
};

const GOAL_ADJUSTMENTS: Record<Goal, number> = {
  cut: 0.8,
  maintain: 1.0,
  bulk: 1.15,
};
```

A.3 Universal Search Setup (`src/lib/universal-search.ts`)

```
const RECIPE_INDEX = new Fuse(localRecipes, {
  keys: ['title', 'cuisines', 'dietary', 'summary'],
  threshold: 0.35,
  ignoreLocation: true,
});

const ARAB_INDEX = new Fuse(arabCuisine, {
  keys: ['name', 'region', 'ingredients'],
  threshold: 0.35,
});

const SUGGEST_INDEX = new Fuse(allItems, {
  keys: ['title', 'name'],
  threshold: 0.55,
  ignoreLocation: true,
});
```

```
export function suggestClosest(query: string): string | null {
  const hit = SUGGEST_INDEX.search(query, { limit: 1 })[0];
  return hit ? (hit.item.title ?? hit.item.name ?? null) : null;
}
```

A.4 SPA Rewrite (`vercel.json`)

```
{
  "rewrites": [
    { "source": "/((?!.*\\.).*)", "destination": "/index.html" }
  ],
  "headers": [
    {
      "source": "/assets/(.*)",
      "headers": [{ "key": "Cache-Control", "value": "public, max-age=31536000,
immutable" }]
    }
  ]
}
```

A.5 Service Worker Registration (`vite.config.ts`)

```
VitePWA({
  registerType: 'autoUpdate',
  workbox: {
    skipWaiting: true,
    clientsClaim: true,
    runtimeCaching: [
      {
        urlPattern: /^https:\/\/i\.ytimg\.com\//,
        handler: 'CacheFirst',
        options: { cacheName: 'youtube-thumbs', expiration: { maxAgeSeconds: 60 * 60
* 24 * 7 } },
      },
      {
        urlPattern: ({ request }) => request.mode === 'navigate',
        handler: 'NetworkFirst',
        options: { cacheName: 'html', networkTimeoutSeconds: 5 },
      },
    ],
  },
  manifest: {
    name: 'Zaytoun',
    short_name: 'Zaytoun',
    theme_color: '#1f1c18',
    background_color: '#fbf6ef',
    display: 'standalone',
    start_url: '/',
```

```
  },
});
```

Appendix B. Sample Screenshots

This appendix lists the screenshots referenced from the report. The actual image files are stored in the `docs/graduation/screenshots/` folder of the project repository.

File	Description
home-desktop.png	Home page on a desktop browser at 1440 px width
home-mobile.png	Home page on a mobile browser at 390 px width
recipe-detail.png	Recipe detail page with ingredients, steps, and embedded video
arab-cuisine-mansaf.png	Arab Cuisine detail page for mansaf
smart-search-arabic.png	Smart search dropdown with Arabic input
shorts-feed.png	Shorts feed grid with category chips and the in app player open
macro-calculator.png	Macro calculator results panel and PDF download button
films-grid.png	Food films library magazine grid
academy-path.png	Academy learning path with progress indicator
chef-hall.png	Chef hall grid grouped by region
markets-atlas.png	World markets atlas
drinks-library.png	Drinks library split into hot and cold sections
dark-mode-home.png	Home page in dark theme
pwa-install.png	Progressive web app install prompt on Android
lighthouse-mobile.png	Lighthouse audit results on simulated slow 4G

Appendix C. Repository and Deployment Information

Item	Value
Project name	Zaytoun
Production URL	https://zaytoun.online
Local development command	npm run dev
Production build command	npm run build

Hosting provider	Vercel
Domain registrar	Spaceship
External recipe API	Spoonacular
Source repository location	C:\Users\ViVoBooK\Desktop\recipe-app on the developer's machine
Default branch	main

Appendix D. Glossary

Term	Definition
BMR	Basal Metabolic Rate. The number of calories the body burns at complete rest.
Mifflin St Jeor	A standard formula for estimating BMR using weight, height, age, and sex.
Katch McArdle	A BMR formula that uses lean body mass, more accurate when body fat percentage is known.
SPA	Single Page Application. A web app that loads one HTML page and updates the view via JavaScript.
PWA	Progressive Web App. A web app that uses modern browser features (service workers, manifest) to be installable and offline capable.
SNI	Server Name Indication. The TLS extension that lets a server host multiple HTTPS sites on one IP. Some ISPs block traffic based on it.
RTL	Right to left, the writing direction used by Arabic.
Fuzzy search	A search method that finds matches even when the query contains typos or partial words.
Lite embed	A pattern that displays a static thumbnail in place of a heavy iframe until the user clicks play.
dvh	A CSS unit equal to one percent of the dynamic viewport height, which adapts to browser UI changes on mobile.